Journal of Parallel and Distributed Computing **62**, 745–765 (2002)
doi:10.1006/jpdc.2001.1823

Local Stabilizer[1]

Yehuda Afek

Computer Science Department, Tel-Aviv University, Tel-Aviv 69978, Israel
E-mail: afek@math.tau.ac.il

and

Shlomi Dolev[2,3]

Department of Computer Science, Ben-Gurion University, Beer-Sheva 84105, Israel
E-mail: dolev@cs.bgu.ac.il

Received October 25, 1999; accepted January 11, 2002

A *local stabilizer* protocol that takes any on- or off-line distributed algorithm and converts it into a synchronous self-stabilizing algorithm with local monitoring and repairing properties is presented. Whenever the self-stabilizing version enters an inconsistent state, the inconsistency is detected, in $O(1)$ time, and the system state is repaired in a local manner. The expected computation time that is lost during the repair process is proportional to the largest diameter of a faulty region. © 2002 Elsevier Science (USA)

1. INTRODUCTION

This paper presents a method that takes a distributed algorithm and produces a synchronous self-stabilizing version that repairs transient faults locally. The expected stabilization time of the resulting algorithm is linear in the diameter of the largest part of the network that is corrupted. Several mechanisms that take an algorithm as an input and produce its self-stabilizing version automatically have been presented in recent years e.g., [5, 7, 10, 15, 21]. However, none of these is applicable to an arbitrary (on- or off-line) input algorithm and is local, where by local we mean first, that as soon as the system enters a corrupted state, that fact is detected and second that the expected computation time lost in recovering from the corrupted state is proportional to the size of the corrupted part of the network (see [25–27] for further discussion and motivation for locality).

[1]An extended abstract of this paper appeared in the Proceedings of the 5th Israeli Symposium on Theory of Computing and Systems, June 1997 and a brief announcement in Proceedings of the 16th Annual ACM Symposium on Principles of Distributed Computing, August 1997.

[2]To whom correspondence should be addressed.

[3]Partially supported by Israeli Ministry of Science and Arts Grant 6756195.

0743-7315/02 $35.00
© 2002 Elsevier Science (USA)
All rights reserved.

Distributed (synchronous) systems, such as network of workstations, serve users at different sites at different times. Synchronization among processors in a distributed system may be achieved by means of clocks (using an outside entity like GPS). The services that such systems support include information delivery as well as access to computing resources. In such systems, users constantly access and interact with the system and distributed algorithms are used to control and manage the activities. These distributed algorithms have to perform sophisticated on-line operations such as traffic control (e.g., deadlock prevention), resource allocation (e.g., mutual exclusion), consistency maintenance (e.g., topology update). This stands in contrast to classical distributed algorithms that are off-line (one-shot). That is, the off-line algorithm takes an input and computes a specific task (e.g., leader-election, minimum spanning tree construction, coloring, consensus). Processors initiate the algorithm and execute it until the off-line task has been computed. On-line distributed algorithms such as in distributed operating systems, on the other hand, consist of interactive on-line (long-lived) tasks.

The goal of this work is to minimize the recovery time, following the occurrence of transient faults, in large distributed on-line systems. As the number of processors grow the frequency of faults increases and the time between faults reduces. If each local fault requires a global output recovery, the system may end up producing wrong outputs most of the time. Therefore, any fault-tolerant mechanism for large systems has to be both *fast* in reaction and *local*.

Following [21], and unlike most standard self-stabilizing algorithms, the self-stabilizing mechanism presented in this paper has two parts. The first part is a self-stabilizing local inconsistency monitoring and detection mechanism, and the second part is a self-stabilizing repair mechanism. The local inconsistency monitor detects any fault or other inconsistency one round after its occurrence. However, the repair process, that brings the entire system back to a consistent state, may cause parts of the network to freeze for a period of time that is proportional to the diameter of the faulty region of the network. Such a freeze may cause delays in operation at distant parts. For example, if few nodes in the middle of a long path connecting two end-to-end nodes fail and recover, then part of the transmission between the two ends suffers an intermittent delay. In this paper, we suggest a new measure, called the *expected computation time loss* to evaluate the complexity of the over-head introduced by the repairing scheme. Roughly speaking, the expected computation time loss is the expected number of rounds each processor pauses its operation due to the corruption of some region. The expected computation time loss of our repair process is proportional to the diameter of a faulty region. The combination of the monitoring and repairing algorithms form a *local stabilizer* that converts any algorithm to a self-stabilizing algorithm with local inconsistency monitoring and repairing.

1.1. Related Work

Checking the consistency of a global state in a distributed system and resetting the system if an inconsistency is detected has been suggested in e.g., [5, 7, 10, 21]. In [21] the first "self-stabilizing compiler" was introduced, in which a global operation is

used to periodically collect a snapshot of the system and examine its consistency. Monitoring consistency *locally* has been suggested in [5, 9, 18]. For example, to construct a rooted tree, a variable indicating the distance from the root can be used to locally check the consistency of the tree. If (and only if) every processor, p, but the root, has a neighboring processor, q, whose distance is smaller than p's distance, the system is in a consistent state (assuming a uniquely predefined root with distance 0). Methods that check the consistency of algorithms are presented in [5, 9] where they are called *local detection* and *local checking*, respectively. Furthermore, [9] suggested the term local *correcting* for a set of self-stabilizing protocols that bring portions of the system back into a legal state. A predicate on the state of a processor and its neighbors holds for each corrected portion and when the predicate holds for every processor, the system is in a consistent state. Note that while portions are corrected, the activity in the other faulty portions of the system can continue and cause additional portions (that were not faulty) to be effected by the faults. Thus, the local correcting technique does not eliminate the possibility of the faults propagating and corrupting the consistency of all the systems.

Note that in the context of self-stabilizing systems, no (reliable) stable storage is assumed and therefore the standard technique of rolling back to a stored consistent snapshot [24, 28, 30], is not applicable. Two techniques, rollback compiler and resynchronizer compiler, for converting synchronous non-interactive algorithms into self-stabilizing asynchronous algorithms are suggested in [11].

Recently, Dolev and Herman [16] and Herman [20] suggested a way to repair the system configuration following the detection of faults (e.g., topology changes) rather then reinitiating the system to a predefined state (i.e., resetting). The repair is carried out by computing a new consistent state to each faulty processor, that is a function of the current (faulty) system state. In [16], self-stabilizing algorithms (e.g., for coloring and tree construction) that enter a legal state in time that is not a function of the number of processors or the diameter of the graph are presented. A similar approach called *mending* has been suggested in [23] and then extended in [22]. The mending approach is based on the voting technique which is used to repair the system configuration for any algorithm that has a fixed output. This fixed output is a function of the systems' communication graph. The technique presented in [23] copes with transient faults that occur only at the very beginning of the execution. The system may not recover if a fault occurs during the repair process and thus is not self-stabilizing. A technique that copes with less than $n/2$ transient faults is presented in [22], unfortunately this solution is not self-stabilizing since it does not ensure that a consistent configuration is reached when the system starts in an arbitrary state. In [19], the term fault-containing is used for (non-interactive) systems that repair a single fault in $O(1)$ time.

1.2. Contributions of This Paper

In this paper, we present two mechanisms and several new notions for self-stabilization of interactive (on-line) systems. The first mechanism is a local monitoring algorithm that detects inconsistency of any distributed protocol in a *single round*. The monitoring algorithm can be applied both to off- and on-line

distributed tasks. The second mechanism is the local repair mechanism. Our repair mechanism is unique not only because it is local but because it is designed to repair the execution of interactive (on-line) algorithms. That is, roughly speaking, a user interacting with the system would not notice that the system has gone through a failure and recovery. The user would probably notice an intermittent freeze of the system response but would not be able to tell the fault by observing the sequence of inputs/outputs she/he has received. Such a repair is not always possible, e.g., if all the processors in the system have failed some or all the user interactions will have to be reset to some predefined initial value. The combination of our two mechanisms, the detection and recovery, guarantees that the likelihood of such a case would be very small, and in most expected cases the recovery would be seamless. To measure the performances of such a recovery process we introduce a new complexity measure, the *expected computation time loss*. This captures the average time it takes a self-stabilizing algorithm to recover. Intuitively, it measures the amount of time the user would see the system she/he interacts with pausing following a failure where the averaging is done over all possible faulty states into which the system may enter following a fault. I.e., the recovery from only a few faulty states takes a long ($O(d)$) time. In computing the expected computation time loss we assume that a transient fault can change the state of a processor to every possible state with equal probability. (This is a possible interpretation and extension of the approach used in [31].) This property, of low expected computation time loss is achieved by employing an error detection mechanism (e.g., error detection codes) to ensure the detection of most failures before they damage any other components of the system.[4] Starting in a global consistent state that is followed by the occurrence of transient faults, that corrupt the state of several processors, the repair procedure ensures that the expected computation time loss by any processor in the network is proportional to the diameter of the faulty region. We call the combination of the two mechanisms, the monitoring algorithm and the repair mechanism, *local stabilizer*.

Our local stabilizer is general and not tailored to a specific algorithm or task. Naturally, general schemes may require more resources (time and space) than a scheme tailored to a specific task. In this work, we minimized the time complexity and use space and communication liberally.

In Section 2, our definitions and model of distributed system are outlined. The monitoring and repairing algorithms are presented in Sections 3 and 4, respectively. Conclusions are given in Section 5.

2. DISTRIBUTED SYSTEM

We consider a distributed system with n processors, each residing on a distinct node of the system's *communication graph* G. The *diameter* of the graph G is denoted by d. Processors communicate by exchanging messages in both directions of each *link* of the communication graph. Each processor is viewed as a state machine. A

[4] In fact we may relax the assumption that a transient fault changes state with equal probability, as long as a transient fault corrupting a state can be detected with arbitrary high probability by the error detection mechanism.

configuration, C, of the system is a vector of states one per processor (the terms system state and configuration are identical).

In the *synchronous* mode of communication, a global clock that generates an infinite sequence of pulses is connected to all the nodes in the network. The time interval between two consecutive pulses of the clock is a *round*. At the beginning of each round, each node decides, according to its state what messages to send and on which links to send them. Each node then receives any messages sent to it by any of its neighbors in this round and receives any input received from its local user (host) in this round.[5] Each node uses its state, the received messages and inputs to decide on its next state.

Let C^t denote a global system configuration at time t, which is also the configuration at the beginning of round t. C^t is a vector $\{c^t_{p1}, c^t_{p2}, \ldots, c^t_{pn}\}$ of the state of each processor at time t. In addition, with each round t we associate a vector $I^t = \{i^t_{p1}, i^t_{p2}, \ldots, i^t_{pn}\}$ in which each element i^t_p is the external input to the corresponding processor at round t.

The *transition function* of processor p, denoted F_p, maps a new state, c^t_p, from c^{t-1}_p, i^{t-1}_p; and the messages received from its neighbors in round $t-1$. In the sequel, the transition is formally written as: $c^t_p = F_p(c^{t-1}_p, i^{t-1}_p, c^{t-1}_{q1}, c^{t-1}_{q2}, \ldots, c^{t-1}_{q\delta})$, where $q1, \ldots, q\delta$ are p's neighbors. In fact, the transition functions of the processors define the source algorithm to be monitored and repaired by our scheme. Note that the state of a neighbor qi that does not send a message to p in round $t-1$ does not affect F_p though we included it in the formal notation.

An *execution* of an interactive system is an infinite sequence of pairs $E = (C^1, I^1), (C^2, I^2), \ldots$ such that for $i > 1$ state c^i_p of any processor p is the result of applying F_p on p's input and on the states of p and its neighbors in C^{i-1}.

Define a *task* by the set of its legal executions, LE, such that any suffix of an execution in LE is also in LE. A global system configuration C is *consistent* (or *legal*) with respect to task LE if any execution of the system starting from configuration C is a legal execution in LE. We assume that a predicate $\mathcal{L}(C)$ that for each configuration C determines whether C is a consistent configuration or not, is available. In the sequel we use $\mathcal{L}(C)$ as part of our monitoring scheme.

We consider faults to be instantaneous transitions. That is, a fault is an instantaneous transition that takes the system from a configuration C to configuration C' by modifying the states of a subset of the processors, these are the *errant* processors. Then the execution continues without additional faults. We show that in such executions the system is guaranteed to behave correctly in time proportional to the maximal diameter of a corrupted region.

2.1. Requirements and Complexity Measures

We have two requirements: *self-stabilization*, and *fault resiliency*, and two complexity measures: *stabilization time* and *expected computation time loss*.

[5] In fact a local user may interact with a node during the entire round. The assumption that the inputs from the local user arrive together with the messages from neighboring nodes (and causes a state change of the node) is used only to simplify the presentation.

Let us start with a few definitions that are necessary for the definition of *fault resiliency* and for the other complexity measures.

An essential and integral part of our complexity analysis is the following assumption on the fault model: faults in our system take a processor from a legal state c, with equal probability, to another state in that processor state space. This fault model captures specific types of transient faults, transient faults that are not malicious (unlike say, Byzantine faults).

DEFINITION 1. Processor p is *errant* relative to a fault that takes the system from configuration C to configuration C' if the states of p in C and C' are different.

DEFINITION 2. The probability of a fault taking the system from configuration C to configuration C', due to a fault that changes the states of k processors, $p_1, p_2, \ldots, p_k$, is $\Pi_1^k(1/|s_i|)$, where s_i is the state space of processor p_i.

DEFINITION 3. An *errant region* in an errant configuration is a maximal connected component of processors that are errant.

While a subset of errant processors recovers from a fault the other non-errant processors may continue their legal operation, as if there had never been any fault. Yet, if the computation at the non-errant processors is affected by the errant processors then the non-errant processors should pause and wait until the recovery process has been completed. Our distributed algorithm uses a boolean variable called *pause* for every processor. The variable *pause* is set to true whenever the processor stops operation due to the occurrence of a fault (somewhere in the network).

DEFINITION 4. A *paused processor* is a processor whose *pause* variable is set to true. The states in which a processor is *paused* are called *paused* states.

Intuitively, our goal is that the sequence of states of a processor, after removing the states in which it is *paused*, is a legal and consistent sequence of states. We use the pause variable to define $\bar{c}_p|NP$ and $\bar{i}_p|NP$ as follows:

DEFINITION 5. Let $\bar{c}_p(E) = c_p^1, c_p^2, c_p^3, \ldots$ ($\bar{i}_p(E) = i_p^1, i_p^2, i_p^3, \ldots$) be the sequence of states (inputs, respectively) of processor p in execution E.

DEFINITION 6. Let $\bar{c}_p(E)|NP$ be the sequence that is obtained by removing all the *paused* states in $\bar{c}_p(E)$. Similarly, the sequence, $\bar{i}_p(E)|NP$ is the sequence of inputs that is obtained by removing all the inputs i_p^j such that c_p^j is removed from $\bar{c}_p(E)$ in obtaining $\bar{c}_p(E)|NP$.

Let us now state the requirements and complexity measures:

R1. *Self-stabilization:* Every system execution that starts from any arbitrary configuration eventually reaches a consistent configuration.

C1. *Stabilization time:* The worst-case number of rounds, over all possible executions, starting from any arbitrary configuration, that it takes to reach a consistent configuration.

R2. *P-fault resiliency:* Intuitively, a system is *P-fault resilient*, $1 > P > 0$, if despite a fault, with probability P processors continue the same execution, i.e., $\bar{c}_p|NP$ is a legal sequence from a legal execution. Formally, given a consistent configuration C, define $E(C)$ to be the set of all executions together with a probability for each execution. Each element $E \in E(C)$ corresponds to different fault transitions and its probability is defined according to Definition 2, C, and C', the first configuration in E. Let $E \in E(C)$, and let $\bar{c}_p = c_p^1, c_p^2, c_p^3, \ldots$ ($\bar{i}_p = i_p^1, i_p^2, i_p^3, \ldots$) be the sequence of states (inputs, respectively) of processor p in E. Then, a system is *P-fault resilient*, if with probability at least P, for every C and every processor p, $\bar{c}_p|NP$ and $\bar{i}_p|NP$ are sequences that appear in a legal execution $E \in LE$.

C2. *Expected computation time loss:* We use the *paused* states to measure the computation time loss. An errant configuration C' can be reached from legal configurations, due to the occurrence of fault, with certain probability e. Let l be the number of rounds during which a processor is *paused* starting in an errant configuration. The *expected computation time loss* is $\Sigma e \cdot l/(\Sigma e)$, where the summation is over all the errant configurations.

Note that we require (in our first requirement R1) that the system will self-stabilize even in the executions in which $\bar{c}_p|NP$ or $\bar{i}_p|NP$ do not appear in a legal execution.

3. LOCAL MONITORING

Intuitively, the local monitoring part works as follows: any algorithm is executed in a general framework that is similar to a full information algorithm. Nodes broadcast their full state and input information to their neighbors and each node keeps a record of all the information available about its neighborhood up to d rounds in the past. Roughly speaking, the record (view) of a processor and the records (views) received from its neighbors are consistent if they have the same value for each processor and they reflect the transition functions of the algorithm. An inconsistency is detected whenever the view of a node and the views collected from neighbors have different values for a state of a processor or when the state transitions in the views do not reflect the transition function.

The next definition uses the term *base state* for the state portion of the algorithm that does not include the auxiliary variables that we introduce for recording snapshots and monitoring.

Definition 7. *Partial snapshot at processor p to distance l at time t* denoted $VI_p^t[l] = (V_p^t[l], I_p^t[l])$, is a snapshot of the l-neighborhood of p at round $t - l$; where $V_p^t[l]$ is the collection of base states and $I_p^t[l]$ is the collection of inputs of all the processors in the l-neighborhood of p at round $t - l$.

In particular, $VI_p^t[0] = (c_p^t, i_p^t)$, is the state of p following the last state transition together with its inputs at round t and $VI_p^t[d]$ is a complete snapshot of the entire system and its inputs at round $t - d$.

DEFINITION 8. Each processor, p, maintains a *pyramid* Δ_p variable of $d+1$ partial snapshots. We denote the *pyramid* of processor p at time t by $\Delta_p^t = VI_p^t[0], VI_p^t[1], VI_p^t[2], \ldots, VI_p^t[d]$ where d is the network diameter.

We use $VI_p^t[j]|q$ to denote the state and input of processor q in $VI_p^t[j]$, note that $VI_p^t[j]|q$ is defined only for processors within distance j from p.

DEFINITION 9. For every two neighboring processors p and q and every $0 < j \leqslant d$ we define the *shared portions* of the system snapshots and input records of $VI_p^t[j]$ and $VI_q^t[j]$ to be the states and inputs of processors r such that r appears in both $VI_p^t[j]$ and $VI_q^t[j]$.

In every round t each processor p communicates Δ_p^t to its neighbors and p constructs a new pyramid for the current t, namely constructs Δ_p^t from Δ_p^{t-1} (see the code in Fig. 1).

THEOREM 3.1. *If in the beginning of round t the system configuration is not a configuration reached by d steps of the algorithm, from a configuration that satisfies the predicate $\mathcal{L}$, and during round t no message is corrupted, then the monitoring algorithm (that appears in the upper part) of Fig. 1 ensures that inconsistency is detected by at least one processor at the beginning of round $t+1$.*

Code of Monitoring and Pyramid update for processor p at round t

```
Let {q1, q2, ..., qδ} be the neighbors of p
    send Δ_p^t to p's neighbors
    receive Δ^t_{[q1,...,qδ]} from p's neighbors
/* Checking consistency: */
    (M1:) Verify that for 0 < j ≤ d V_p[j − 1]|p = F_p(VI_p[j]|p, V_p[j]|q1, V_p[j]|q2, ···).
                    /* That is, the state component of VI_p[j − 1]|p is the state that */
                    /* p would be in when making a transition from the state in */
                    /* V_p[j]|p, after receiving the local input in */
                    /* I_p[j]|p and receiving messages from neighbors q */
                    /* that were in state V_p[j]|q. */
    (M2:) Verify that L(V_p[d]) is true.
                    /* i.e., V_p[d] is consistent according to the task/protocol */
                    /* specifications. */
    (M3:) For every 0 < j ≤ d and every neighbor q,
                verify that the shared portions of VI_p[j] and VI_q[j] agree.
    if either of M1,M2 or M3 is false then
            INCONSISTENCY DETECTED /* A trigger for the repair process */
    else
/* Updating Δ_p (adding 1 to t): */
    (U1:) V_p^t[0]|p = F_p(VI_p^{t−1}[0]|p, V_{q1}^{t−1}[0]|q1, ..., V_{qδ}^{t−1}[0]|qδ). */
                    /* That is, the state component of VI_p^t[0]|p is the state */
                    /* obtained by applying the algorithm's transition function */
                    /* on the local input of p and the states of p and every of */
                    /* its neighbors at round t − 1. */
    (U2:) For 0 < j ≤ d, VI_p^t[j] is constructed from VI_p^{t−1}[j − 1] and VI_q^{t−1}[j − 1]
                for every q ∈ {q1, q2, ..., qδ}.
                    /* VI_p^{t−1}[j − 1] is extended into VI_p^t[j] by adding */
                    /* the corresponding elements from VI_q^{t−1}[j − 1] that are at */
                    /* distance j − 1 from q and at distance j from p */
```

FIG. 1. Monitoring and Pyramid update.

Proof. By the transitivity of equality, Step M3 of the monitoring steps ensures that if no inconsistency is detected then for every processor q, $VI_q[d] = VI_p[d]$. By Step M2, $V_p[d]$ is a consistent state. Step M3 also ensures that the processors have the same values, in the overlapping portions of their pyramids, for the inputs received by the processors. We now prove that inconsistency is detected if the *current* system state is not the state reached from $VI_p[d]$ in d rounds during which the processors received the inputs that appear in the pyramids. Assume towards a contradiction that there exists $1 \leqslant j < d$ such that the state of processor p in $VI_p[j]$ is not implied by $VI_p[d]$ and the inputs that appear in the pyramids of the processors. Let k be the largest such j. By the choice of k, the states of p and its neighbors $q_1, q_2, \ldots$, in $VI_p[k+1]$ are reached from $VI_p[d]$ by the operations of the algorithm when the inputs are as in the pyramids. By our assumption $V_p[k]|p \neq F_p(VI_p[k+1]|p, V_p[k+1]|q_1, V_p[k+1]|q_2, \ldots)$. This fact is discovered by p in Step M1—a contradiction. ∎

The update procedure of processor p appears (in the lower part) in Fig. 1. The input for the update procedure of p consists of the pyramids of p and the pyramids of p's neighbors $q1, q2, \ldots, q\delta$ produced in the previous round and the local input of p during the previous round stored in $I_p^{t-1}[1]$.

DEFINITION 10. A *valid pyramid* $\Delta_p^t = VI_p^t[0], VI_p^t[1], VI_p^t[2], \ldots, VI_p^t[d]$ of an execution $E \in LE$, $E = (C^0, I^0), (C^1, I^1), \ldots, (C^d, I^d)$, is a pyramid of snapshots $VI_p^t[0], VI_p^t[1], \ldots, VI_p^t[d]$ such that $VI_p^t[k]$ is a (distance k) snapshot of C^k and I^k.

THEOREM 3.2. *Starting with a valid pyramid for an execution $E \in LE$ for every processor p, the update procedure (in the lower part) of Fig. 1 produces a valid pyramid of the execution $E' \in LE$ for every processor p such that when (C^d, I^d) is removed from E then E is a prefix of E'.*

4. REPAIRING

In the repairing scheme, the pyramids of partial snapshots are used in order to regain the consistency of the errant regions. The snapshots in the pyramids of the non-errant regions are diffused into the errant regions, until each errant processor reconstructs a pyramid which is consistent with its neighbors. When a processor has a full pyramid which is consistent with all its neighbors, it may continue operation as if there were no faults. However, this method may fail to bring the system into a legal consistent state if following a fault there are two or more components of the network in which the processors are consistent within themselves (there is a configuration in *LE* with the combination of their states) but the configuration that includes all components is not a configuration in *LE*. E.g., following a fault, half of the network may claim that it is day time and the other half may claim it is night. In such a case, there may be no other solution for resolving the conflict between the two (or more) internally consistent components but to use a reset procedure that reinitiates the entire system. The complexity of

such a reset is $O(d)$ stabilization time and may violate the P-fault resiliency requirement. Moreover, the existence of such a reset procedure (which is invoked by the processors) and the fact that transient faults may cause processors to enter an arbitrary state, may imply that the reset procedure can be erroneously invoked even if a small portion of the system experiences faults (i.e., by erroneously moving a processor into the state in which it invokes the reset procedure).

Our approach around the above scenario is to introduce a mechanism that drastically reduces the probability of a fault taking the system into another consistent state. That is, we assume that a fault at a processor takes it into another state in its state space in a random way with uniform distribution over the state space. The idea is then to increase the state space with many junk states making the probability of a fault taking the processor into a legal state very small. This idea is implemented by using error detection codes to encode the states of processors (we assume that a state of a processor can be represented by a binary string). At every step of the algorithm the state encoding is checked and if an error is detected, that processor is declared errant. The probability of a fault taking a processor into a consistent state can be tuned arbitrarily small by choosing a large enough error detection code.[6] Note that this mechanism is used only to reduce the probability of a global reset following a local fault, the self-stabilization property of our method does not depend on it.

To ensure the self-stabilization property of the *local stabilizer* we add one more mechanism that works like a "watch-dog" — safety fall-back mechanism (see e.g., [1, 19] for similar approaches). In the repair process (given below) errant processors acquire their state back from the snapshots maintained at neighboring non-errant processors. However, it could be for example, that there are no non-errant processors in the system and that the errant processors should start a global reset procedure. To ensure that in such situations the errant processors start a reset procedure we add this "watch-dog" mechanism that works in parallel to the repair process as follows: whenever a fault is detected the processors start to count in a special counter called *repair counter*. If the *repair counter* value reaches $2d$ and the repair process is not complete then a reset procedure is invoked (in the sequel we argue that the maximum time it should take to repair an errant region is $2d$). Note that the "watch-dog" mechanism is self-stabilizing and that the repair counter is also encoded with enough error detection bits to minimize the probability of erroneously reaching the wrong value.

Note that if a fault is not detected (e.g., a fault that moves a processor into a locally legal state) then the consistency monitoring mechanism would detect an inconsistency (assuming the fault has moved the system into a globally inconsistent state, otherwise the fault has moved the system into a globally legal state where there is nothing to do). The consistency monitoring mechanism is integrated in the code for the repair process presented in the next section.

[6] Because a fault takes a processor into an arbitrary state, error correcting codes cannot be used for reconstructing the state before the transient fault.

4.1. The Repair Process

The repair process is the procedure by which errant processors regain their consistent state following the detection of an error by the error detection code. It runs in parallel to the "watch-dog" mechanism as explained above. Note that if processors detect an inconsistent state, i.e., neighboring processors having each a locally legal state but mutually inconsistent, then they invoke a global reset procedure. In such cases we do not count on the repair process to regain consistency.

Following a fault, it is important for an errant processor to reconstruct its pyramid using the information in the pyramids of the non-corrupted portions of the system and the transition function F. However, processor p should use F_p to determine a missing state in its pyramid only when no (conflicting) information on its state is about to arrive. To ensure that no such information exists we use two variables for each processor: a *time counter* and a *repair counter*. The *time counter* is incremented by 1 in every pulse in which the processor is not *paused*. For ease of description we first assume that the *time counter* is unbounded, then we show a way to bound the *time-counter* value.

In the consistency monitoring mechanism each processor checks that in each round its *time-counter* value is the same as the *time-counter* values of its neighbors. As will be seen in the sequel, during the recovery process it could be that neighboring processors have different *time counters*. Let x and y, $x > y$, be the values of the time counters of processors p and q, respectively. In such a case, the consistency monitoring mechanism checks that $VI_p^t[l + (x - y)]$ and $VI_q^t[l]$ of the pyramids (that correspond to the same round) are consistent for every $0 \leqslant l + (x - y) \leqslant d - (x - y)$.

Upon detecting a fault a processor empties its pyramid thus signaling to its neighbors that it is errant. We use *nil* as a special value that is used as a flag for missing information. A pyramid is *full* if it does not contain a *nil* value. A pyramid is *empty* if it contains only *nil* values. In the sequel we use the term emptying the pyramid for assigning a *nil* value to each of the pyramid entries. The goal of the repair process is to reconstruct for each errant process a full pyramid that is consistent with the pyramids of its neighbors, i.e., in which the pyramids are consistent and the *time counter* has the same value as its neighbor's *time counter*s.

As was previously mentioned, each processor has a *paused* variable. The values of the *repair counter* and the *reset counter* (which are part of the reset procedure as explained below), together with the value of the *paused* variable and the contents of the pyramid defines the state of a processor to be at any point of time in one of four states.

operating: (*paused* = false, *reset counter* = 0)
: A processor that operates correctly and has detected neither a fault nor an inconsistency in the last round.

errant: (*paused* = true, *reset counter* = 0, non-full pyramid)
: A processor that has detected a fault by the error detection code, in the recent past and its pyramid is still incomplete. That is, its pyramid could be already partially reconstructed but still has *nil* values.

paused: (*paused* = true, *reset counter* = 0, full pyramid)
A *non-errant* operating processor enters a *paused* state at time t if either one of its neighbors is *errant*, or *paused* in time $t-1$. In general, a processor goes out of the pause state when all its neighbors have full pyramids that are consistent with its own and the values in the *time counters* variables are the same.

reset: (*reset counter* $\neq 0$)
A processor enters a reset state either when it detects that a reset needs to be performed, or when one of its neighbors does. In the sequel we describe the reset procedure and how processors go out of it into an operating state.

Intuitively, the repair process is very simple, errant processors empty their pyramids and clear their *time counter* upon the detection of a fault by the error detection code. Thereafter, each errant processor receives in each round the pyramids of its neighbors (even if these are also empty) and takes from these pyramids as much information as it may in order to reconstruct a pyramid of its own with the largest *time-counter* value seen in any of its neighbors. As a cluster of processors may fail together, this process repeats at each errant processor until it has a full pyramid which is consistent with the full pyramids of its neighbors or until the "watch-dog" mechanism fires. A *non-errant* processor that has an *errant* neighbor or a *paused* neighbor *pauses* by itself and freezes its pyramid and *time counter* until all its neighbors are consistent and ready to take a step forward.

In reconstructing its pyramid a processor finds its recent history in its *non-errant* neighbors. However, if all processors in radius r around a processor are *errant*, then its states in the $r+1$ rounds before the occurrence of the fault are lost. In particular, there is no record of the inputs of the user for these rounds. If the *errant* processor has had the missing inputs then it could recompute the lost states based on the states obtained from the *non-errant* processors and the inputs. However, if the processor is supposed to recompute exactly the same states as before the failure, then it must gain the inputs it received before the failure. There are two possibilities to handle the missing inputs, when non-volatile memory is not available:

1. Ask the user to resupply the missing inputs.[7]
2. Assume these inputs were a special default *nil* value. (In which case the system would return to a globally consistent state but not necessarily the exact same state as before the failure.)

Which of the two methods is available is orthogonal to our algorithm, and in the sequel we will assume that either one of these options is available.

The "watch-dog" mechanism is implemented by using a *repair counter* that counts the number of rounds since a fault is detected by the error detection mechanism. Any processor whose repair-counter value reaches $2d$ checks that the repair process has finished: all pyramids are full and the monitoring rules do not detect inconsistency. If this test indicates that the repair process was not successful the reset counter is set

[7] If the user is another program, then in fact a non-volatile memory to store the missing inputs should exist.

to 1. This assignment triggers a global reset operation, as follows: in every pulse, each processor sets its reset counter to be the smallest non-zero reset counter plus one. As we show in the sequel the above operation ensures that all the reset counters hold simultaneously the value $2d$ in a certain round, triggering a simultaneous initialization of the system.

Note that the repair counter and the reset counter themselves are subject (with a very small probability) to (undetected) transient faults. Such a fault may cause (with a very low probability) a processor to assign 1 to its reset counter and trigger a global reset.

The formal description of the algorithm appears in Fig. 2. We also describe the algorithm by listing several rules. When the round starts each processor p first checks and executes the error detection rule then p sends its pyramid to its neighbors. Processor p receives the pyramids of its neighbors and uses its pyramid and the received pyramids to execute the other rules. We say that a rule is *applicable*

```
Code performed by processor p, in each round do:
   /* Error-Detection Rule: */
01     if (Error-detection(state) = true) then
02         repair-counter := 1
03         pyramid := nil
04         time-counter := nil
05         paused := true
06     fi
07     send pyramid, and all other state variables to each of the neighbors
08     receive pyramids and variables from each of the neighbors
   /* Reset Propagation Rule: */
09     R := { reset-counters of all the neighbors } ∪ { reset-counter}
10     if (∃l ∈ R s.t. l > 0) then
11         r := minimum {l ∈ R and l > 0}
12         if (r ≥ 2d) then
13                 pyramid := initial value
14                 time-counter := 0
15                 reset-counter := 0
16                 repair-counter := 2d
17         else reset-counter := r + 1
18         fi
19     else                                   {the rest of the code is performed only if there is no reset.}
   /* Repair Propagation Rule: */
20         P := { repair-counters of all the neighbors } ∪ { repair-counter}
21         rp := minimum {l ∈ P}
22         if (rp ≥ 2d) then
23                 repair-counter := 2d
24         else repair-counter := rp + 1
25         fi
   /* Paused Rule: */
26         if (exists a neighbor whose time-counter is smaller than p's) or
27             (pyramid or a neighbor pyramid is not full) then
28             paused := true
29         else
30             paused := false
31         fi
   /* Monitoring Rule: */
32         T := { time-counters of all the neighbors } ∪ { time-counter}
33         if ((time-counter ≠ nil) and (∃l ∈ T s.t. | time-counter −l| > 1)) or
34             (M1, M2 and M3 applied to the non nil portions detect inconsistency) or
35             ((repair-counter = 2d) and ((there exists nil state in pyramid) or
36             (minimum{T}≠ maximum{T})) then                                   { watch dog}
37             reset-counter := 1
38         fi
   /* Updating Rule: */
39         Use information in the pyramids of your neighbors to replace nil values
40         if j is the maximal value for which the state of p in VI_p[j] is nil and
41         repair-counter > j then
42             recompute the state of p in VI_p[j]
43         fi
44         if repair-counter= 2d then apply U1 and U2                           { update}
45     fi
```

FIG. 2. The operations performed by each processor.

if it causes a value change in some variable. In case the reset propagation rule is applicable the rest of the rules are not used. Otherwise, the remaining rules are checked and executed sequentially one after the other. Next, we list the rules together with the lines of code that correspond to each rule.

Error detection rule (Lines 01–06): If the error detection code indicates that there is an error then the process sets the *repair counter* to 1, sets its pyramid and *time counter* to *nil* values, and sets its *paused* variable to *true*.

Reset propagation rule (Lines 09–18): Let $\mathcal{R}$ be a set of *reset counter* values received by processors from their neighbors together with the value of the *reset counter* of the processors themselves. If there exists a non-zero value in $\mathcal{R}$ and the minimal such value r is less than $2d$ then the processor assigns $r + 1$ to its *reset counter*. Otherwise, if the value of r is greater than $2d - 1$ then the processor assigns its state including its pyramid to a predefined initial state. In this initial state the values of the *reset counter*, the *repair counter* and the *time counter* are zero.

Repair propagation rule (Lines 20–25): Let $\mathscr{P}$ be a set that consists of the values of the *repair counter*s received by a processor from its neighbors together with the value of the *repair counter* of the processor itself. If rp, the minimal value in $\mathscr{P}$, is less than $2d$ then the processor assigns $rp + 1$ to its *repair counter*. In case the value of rp is at least $2d$ then the processor assigns $2d$ to its *repair counter*.

Paused rule (Lines 26–31): A processor p with a full pyramid that receives a non-full pyramid or a *time counter* of value less than its own *time counter*, from one of its neighbors, assigns *true* to its *paused* variable. Processor p with a full pyramid assigns *false* to its *paused* variable when p receives from each of its neighbors a full pyramid and a *time counter* with value greater than or equal to its own *time counter*.

A processor with *true paused* variable and full pyramid does not change its pyramid contents nor does it change the value of its *time counter*.

Monitoring rule (Lines 32–38): The monitoring rule detects inconsistency and triggers a reset. A reset is triggered when either the value of the *time counters* are inconsistent, or when there exist two different values for the same state in the system, and when the repair process is not complete in $2d$ rounds.

A processor uses the monitoring steps M1–M3 in Fig. 1 for every non-*nil* input of its pyramid and the pyramids of its neighbors. The monitoring steps take into account the shared portions of the pyramids defined according to the *time counter* of the pyramids; M1 checks whether the states of processor p, that can be computed from the non-*nil* values in p's pyramid are obtained by the transition function F_p applied to the appropriate non-*nil* values in the pyramid of p. M2 verifies that $\mathcal{L}(V_p[d])$ is true in case $V_p[d]$ does not include *nil* values. M3 checks equality of non-*nil* states and inputs that are related to the same processor at the same time according to the *time counters* of the processors to which these pyramids belong. The partial snapshot $VI_p[j]$ and the partial snapshot $VI_q[j-1]$ contain information on

the states of the processors and inputs that are related to the same time if the value of the *time counter* of p is greater than the value of the *time counter* of q by 1. For example, the state $V_p[j]|x$ in the pyramid of p should be equal to $V_q[j-1]|x$ in the pyramid of q if the *time counter* of p is greater than the *time counter* of q by 1.

If either of the above monitor steps detects inconsistency then processor p sets its *reset counter* to 1.

Updating rule (Lines 39–44): Every *nil* value (state or input) in the pyramid of a processor p, is replaced by a non-*nil* value that is received in a pyramid communicated by a neighbor. In particular, the time counter is assigned by the largest time counter of a neighbor.

Similar to the monitoring rule, the correspondence of values in a pyramid that is received from a processor q, and the pyramid of p is defined by the value of the *time counters* of p and q. The partial snapshot $VI_p[j]$ and the partial snapshot $VI_q[j-1]$ contain information on the states of the processors and inputs that are related to the same time if the value of the *time counter* of p is greater than the value of the *time counter* of q by 1.

In addition, a processor p computes its state in $V_p[j]$ using the transition function F_p, if the states of p and p's neighbors appear in $V_p[j+1]$, the state of p in $V_p[j]$ is *nil*, and the value of the *repair counter* of p is greater than j.[8] For example, a processor p with a *repair counter* value 2 can recompute its state in $V_p[1]$ if the state of p and the neighbors of p appear in $V_p[2]$ of the pyramid of p and the state of p in $V_p[1]$ is *nil*. If inputs are missing for computing the new state of p by F_p, then p requests the user for the missing inputs or uses *nil* inputs, depending on the system choice for handling missing inputs.

Finally, the update rules U1 and U2 of Fig. 1 are applied in case the repair procedure is completed.

Before we present the correctness and complexity proofs let us add a few lines to explain intuitively the way the counters are used. We start with the reset counter.

Consider a fault-free execution that starts in an arbitrary configuration. In a system configuration in which all the reset counters are zero there is no active reset. If a processor assigns 1 to its reset counter as a result of detecting inconsistency (line 37 of the code) then a successful reset will be invoked: the idea is that the minimal reset value propagates while incremented by 1, reaching every processor in the system in d rounds (where d is the diameter of the system). Thus, in d rounds a configuration C is reached in which the reset counter of every processor is non-zero and is less than or equal to d (processors always choose the minimal reset counter value). No processor assigns 1 to its reset counter in the d rounds that follow C (the else in line 19 of the code is not executed). Therefore, if the minimal value of a reset counter in C is m (m must be less than or equal to d) then the reset counter of every processor holds $m+d$, d rounds following C. In other words, the processors are synchronized and ready to execute reset simultaneously once the value of the reset counters is $2d$. This is in fact the key idea for our simple reset mechanism.

[8] The value of the *repair counter* ensures that the recomputed state of p does not appear elsewhere in the system.

Now we are ready to explain the task of the repair counter. The repair counter is used when the error-detection code indicates an error. Again in a system configuration in which all the repair counters are zero there is no active repair. Once a processor assigns 1 to its repair counter due to the error detecting code the repair process starts. The repair process itself is based on propagating state information from non-effected regions to effected region. To allow existing state information to reach the infected region we count until the repair counter reaches $2d$. At first glance it may seem that a repair counter that is bounded by the value d is sufficient, however, the repair process also involves reconstruction of the pyramids using the data received and spreading the new reconstructed information in the system, which requires additional d rounds. Once the repair counters of the processors reach $2d$ the processors check that the repair process is finished (there are no nil values in the pyramids) and the system is in a consistent state. One more important task of the repair counter is the indication that no more information is available and recomputation of states should be invoked: when a processor is waiting for information that is related to one of the top layers of its pyramid, say the third, it can be sure that no such information is available if its repair counter reaches the value 3. This observation is used in lines 41 and 42 of the code.

The task of the time counter is to relate state information stored in neighboring pyramids. Since processors can be *paused* it is possible that one processor will have a time counter that is different by one from its neighbor. This information is used in the repair process when nil values are replaced with state information.

The reader may ask what is the influence of (reset, repair or time) counter corruption that is not identified by the error detection — the answer is identical to the case in which the state information is corrupted and not identified, namely, an arbitrary state is reached from which the system stabilizes — i.e., reset is invoked as we next prove.

4.1.1. The correctness and complexity proofs

LEMMA 1. *Any fault-free execution that starts in an arbitrary configuration that is immediately followed by the assignment of* 1 *to a* reset counter *enters a consistent global state within at most* $3d$ *rounds.*

Proof. The reset propagation rule ensures that d rounds following the assignment of the value 1 to a *reset counter*, it holds that the value of every *reset counter* is greater than 0 and less than $d + 1$. Thus, the rules that follow the reset propagation rule are not executed following the first $d - 1$ rounds. In particular, the value of the *reset counter* is not assigned by 1 in the monitoring rule.

Let y be the minimal value of a *reset counter* d rounds following the assignment of the value 1 to a *reset counter*. The value of y propagates to every processor (while incremented) in the next d rounds. Thus, within $d - y + 2d$ rounds the processors simultaneously assign a predefined initial state that is consistent. ∎

The next theorem proves that our system is self-stabilizing.

THEOREM 4.1. *In every fault-free execution that starts in an arbitrary configuration the system reaches a consistent configuration within at most* $5d + 1$ *rounds.*

Proof. First, we show that if no processor assigns 1 to its *reset counter* during the first $2d$ rounds of the execution then following these $2d$ rounds a configuration, C, is reached in which the value of every *reset counter* is zero and the value of every *repair counter* is $2d$: the minimal non-zero value of a *reset counter* is incremented by one in every round until it is $2d - 1$. One round following the configuration in which the minimal non-zero value of a *reset counter* is $2d - 1$ all the values of the *reset counters* must be zero. By our assumption, no processor assigns 1 to its *reset counter*, therefore once the values of all the *reset counters* are zero these values are not changed.

As for the values of the *repair counters*, note that no processor assigns 1 to its *repair counter* in a fault-free execution, since no error is detected by the error detection code in a fault-free execution. Thus, the smallest non-zero value of a *repair counter* is incremented in every round until the value $2d$ is reached.

If no processor assigns 1 to its *reset counter* in the round that follows C then all the pyramids are full and the values of all *time counters* are equal. By Theorem 3.1 if no processor assigns 1 to its *reset counter* in the round that follows C, then c is a consistent configuration. Thus, if C is not consistent then at least one processor assigns 1 to its *reset counter* during the first $2d$ rounds of the execution. Once a processor assigns 1 to its *reset counter* then by Lemma 1 a consistent configuration is reached within additional $3d$ rounds. ∎

The next theorem proves that the algorithm fulfills the P-fault resiliency property. The proof is for the case in which *nil* inputs are used by *errant* processors until they resume operation; we note that the case in which the user resupplies the missing inputs is simpler.

THEOREM 4.2. *For every $P < 1$ there exists an error detection code such that the fault resiliency requirement holds with probability P.*

Proof. By an appropriate choice of the error detecting code, where the amount of redundancy used by the error detecting code is a function of P, all the processors that experience transient faults detect the occurrence of the fault with probability greater than or equal to P.

We show that the non-*paused* state sequence of all the *non-errant* processors appears in a legal execution. The legal execution we choose is the one that starts in the full configuration stored in the (base of the) pyramids of the *non-errant* processors when the faults occur. Then the execution continues as if there are no faults and the input of every *non-errant* process is identical to the input that appears in their pyramids when the faults occur. The inputs of every *errant* processor is the input stored for it in the pyramids of the *non-errant* processors, if such input exists, or *nil* otherwise. The above execution starts in a consistent state, namely, the full snapshot in the base of the pyramids, and continues with possible inputs changing states according to the program of the processors, thus reaching a consistent state. The execution continues from this consistent state normally, receiving inputs from the users and acting according to the programs.

Clearly, the above execution is a legal execution corresponding to a fault-free execution. Thus, to prove the theorem, it is enough to show that the sequence of the

non-*paused* states of every process during the repairing process is identical to the sequence of its states in the above execution.

One important observation is that the *errant* processors do not recompute their state unless the recomputed state does not exist in the pyramid of the non-*errant* processors. The reason for this is that every *errant* processor recomputes a state in $VI_p[j]$ only when the value of the *repair counter* is at least j, indicating that information from all processors of distance j or less was already used.

Thus, the state transitions of the *errant* and *non-errant* processors fit the execution described above. The *errant* processors are *paused* until they receive the information from the *non-errant* processors, use this information (states and inputs) in these pyramids and continues changing states accordingly. It is also clear for the *non-errant* processors that are *paused* until they receive *time counter*s with values equal to or greater than their own *time counter*. ∎

The next theorem proves that starting in a consistent global state, that is followed by transient faults, the expected computation time loss is proportional to the maximal diameter of an infected region. Our proof is based on the probabilistic nature of a fault that ensures that a legal state processor that experiences a fault (does not reach another legal state and thus) detects the occurrence of the fault. Note however that the system regains consistency also in the case in which the fault is not detected.

THEOREM 4.3. *In every execution in which all the faults occur simultaneously the expected computation time loss is proportional to the maximal diameter of an infected region.*

Proof. By an appropriate choice of an error detecting code every processor that experiences a transient fault detects the occurrence of the fault with probability close to 1. Let i be the maximal diameter of an infected region. Our repair scheme distributes the pyramids of the *non-errant* processor to every *errant* processor within at most i rounds. Then every *errant* processor recomputes a missing state (there are at most i such missing states) in every round until the pyramid is full. Thus, additional i rounds are required. Once the *non-errant* processors receive the pyramids from each of their neighbors they resume operation. Thus, every processor may stop operation for at most $2i$ rounds. ∎

4.2. Bounding the Time-Counter Value

For a self-stabilizing solution it is most important to bound the *time-counter* value, since by the nature of self-stabilization any counter can be started with its upper limit value. We next show that a *time counter* that is incremented modulo $3d$ is sufficient for our purposes. In a consistent configuration the values of the *time counters* are equal, faults are detected so they do not introduce new *time-counter* values. *Non-errant* processor may increment their *time-counter* value while a processor is *paused* by at most d. Thus, in every configuration the maximal number of different *time-counter* values is d. Given two *time-counter* values x and y (both obtained by modulo

$3d$ increments) where $x > y$, we say that x is greater than y if $x - y < (3d/2)$, and otherwise x is smaller than y.

5. CONCLUDING REMARKS AND EXTENSIONS

The amount of communication used for monitoring the consistency of an algorithm can be significantly reduced. The pyramid sent in each step from a processor p to a processor q should contain existing information in the pyramid of q. Thus, p can randomly choose a key and calculates the checksum relative to this key and sends both the key and the checksum to q (similar to the technique proposed in [17]). q will use the received key and checksum to verify (with high probability) that indeed the shared portions of the pyramids are identical. This scheme reduces the communication from p to q to include only the information that appears in p's pyramid but not in q's pyramid (in addition to the key and checksum).

It is interesting to note that a variant of our monitoring algorithm can be applied to an asynchronous system as well. The asynchronous version of our protocol detects inconsistency in a single asynchronous round. The scheme is based on the synchronous solution. Asynchronous pulses are implemented. The pulses are used for monitoring only; the computation of the task is asynchronous. Every processor, p, maintains a *pulse-counter* PC_p. Every processor q repeatedly examines the state of all of its neighbors and detects a fault in case the value of $|PC_p - PC_q| > 1$. Whenever p finds that for each of its neighbors q $PC_p \leqslant PC_q$ then p monitors the system consistency, in a manner similar to the monitoring scheme that appears in Fig. 1, increments PC_p by 1 and updates Δ_p as described in Fig. 1. Furthermore, PC_p can be incremented modulo M, where M is a constant that is larger than the number of processors in the system.

We note that it is possible to use the monitoring technique to trigger the repair process (and not only the identification of an erroneous state by the error detection codes). Our choice not to describe such a version was for the sake of presentation simplicity.

We present a technique to monitor the consistency of interactive tasks, locally, in a single time unit. The inconsistency detection is coupled with repairing procedure to yield an algorithm that locally monitors the consistency of a distributed system and upon detection of inconsistency rapidly repairs the system state in order to regain consistency.

ACKNOWLEDGMENTS

It is a pleasure to thank Moti Yung and the three anonymous referees for helpful remarks. We also thank Antoly Shein for his remarks while implementing our algorithms.

REFERENCES

1. H. Abu-Amara, B. Coan, S. Dolev, A. Kanevsky, and J. L. Welch, Self-stabilizing topology maintenance protocols for highs-speed networks, *IEEE/ACM Trans. Networking* **4**(6) (1996), 902–912.

2. Y. Afek, B. Awerbuch, and E. Gafni, Applying static network protocols to dynamic networks, *in* "Proceedings of the 28th Annual IEEE Symposium on Foundations of Computer Science," pp. 358–370, 1987.
3. Y. Afek and G. M. Brown, Self-stabilization over unreliable communication media, *Distrib. Comput.* **7** (1993), 27–34.
4. Y. Afek and S. Dolev, Local stabilizer, *in* "Proceedings of the 5th Israeli Symposium on Theory of Computing and Systems," June 1997 and Technical Report 97-02, Department of Mathematics and Computer Science, Ben-Gurion University, February 1997.
5. Y. Afek, S. Kutten, and M. Yung, Memory-efficient self-stabilization on general networks, *in* "Proceedings of the 4th Workshop on Distributed Algorithms," pp. 15–28, 1990.
6. A. Arora, S. Dolev and M. G. Gouda, Maintaining digital clocks in step, *Parallel Process. Lett.* **1**(1) (1991), 11–18.
7. A. Arora and M. G. Gouda, Distributed reset, *IEEE Trans. Comput.* **43** (1994), 1026–1038; also *in* "Proceedings of the 10th Conference on Foundations of Software Technology and Theoretical Computer Science," pp. 316–331, 1990.
8. B. Awerbuch, S. Kutten, Y. Mansour, B. Patt-Shamir, and G. Varghese, Time optimal self-stabilizing synchronization, *in* "Proceedings of the 25th ACM Symposium on Theory of Computing," 1993.
9. B. Awerbuch, B. Patt-Shamir, and G. Varghese, Self-stabilization by local checking and correction, *in* "Proceedings of the 32nd IEEE Symposium on Foundations of Computer Science," pp. 268–277, 1991.
10. B. Awerbuch, B. Patt-Shamir, G. Varghese, and S. Dolev, Self-stabilization by local checking and global reset, *in* "Proceedings of the 8th Workshop on Distributed Algorithms," pp. 226–239, 1994.
11. B. Awerbuch and G. Varghese, Distributed program checking: A paradigm for building self-stabilizing distributed protocols, *in* "Proceedings of the 32nd IEEE Symposium on Foundations of Computer Science," pp. 258–267, 1991.
12. K. M. Chandy and L. Lamport, Distributed snapshots: Determining global states of distributed systems, *ACM Trans. Comput. System* **3**(1) (1985), 163–75.
13. E. W. Dijkstra, Self-stabilizing systems in spite of distributed control, *Comm. Assoc. Comput. Mach.* **17**(11) (1974), 643–644.
14. S. Dolev, Possible and impossible self-stabilizing digital clock synchronization in general graphs, *J. Real-Time Systems* **12**(1) (1997), 95–107.
15. S. Dolev, "Self-Stabilization," MIT Press, New York.
16. S. Dolev and T. Herman, Superstabilizing protocols for dynamic distributed systems, *in* "Proceedings of the Second Workshop on Self-Stabilizing Systems," 1995; *Chicago J. Theoret. Comput. Sci.* **3**(4) (Special Issue on Self-Stabilization) (1997).
17. S. Dolev, G. Gouda, and M. Schneider, Memory requirements for silent stabilization, *in* "Proceedings of the 15th Annual ACM Symposium on Principles of Distributed Computing," pp. 27–34, 1996; *Acta Inform.* **36** (1999), 447–462.
18. S. Dolev, A. Israeli, and S. Moran, Self stabilization of dynamic systems assuming only read/write atomicity, *Distrib. Comput.* **7** (1993), 3–16; "Proceedings of the Ninth Annual ACM Symposium on Principles of Distributed Computation," Montreal, pp. 103–117, August 1990; "Proceedings of the First Workshop on Self-Stabilizing Systems," 1989.
19. S. Ghosh, A. Gupta, T. Herman, and S. V. Pemmaraju, Fault-containing self-stabilizing algorithms, *in* "Proceedings of the Fifteenth Annual ACM Symposium on Principles of Distributed Computation," pp. 45–54, Philadelphia, May 1996.
20. T. Herman, Superstabilizing mutual exclusion, *Distrib. Comput.* **13** (2000), 1–17.
21. S. Katz and K. J. Perry, Self-stabilizing extensions for message-passing systems, *Distrib. Comput.* **7** (1993) 17–26; *in* "Proceedings of the Ninth Annual ACM Symposium on Principles of Distributed Computation," Montreal, pp. 91–101, August 1990; "Proceedings of the First Workshop on Self-Stabilizing Systems," 1989.

22. S. Kutten and B. Patt-Shamir, Time-adaptive self-stabilization, *in* "Proceedings of the 16th Annual ACM Symposium on Principles of Distributed Computing," pp. 149–158, 1997.
23. S. Kutten and D. Peleg, Fault-local distributed mending *in* "Proceedings of the Fourteenth Annual ACM Symposium on Principles of Distributed Computation," Montreal, pp. 20–27. August 1995.
24. R. Koo and S. Toueg, Checkpointing and rollback-recovery for distributed systems, *IEEE Trans. Software Eng.* **13** (1987), 23–31.
25. N. Linial, Locality in distributed graph algorithms, *SIAM J. Comput.* **21** (1992), 193–201.
26. A. Mayer, M. Naor and L. Stockmeyer, Local computations on static and dynamic graphs, *in* "Proceedings of the ISTCS'95, Israel Symposium on Theory of Computing and Systems," 1995.
27. M. Naor and L. Stockmeyer, What can be computed locally?, *in* "Proceedings of the 25th ACM Symposium on Theory of Computing," pp. 409–412, 1993.
28. A. P. Sistla and J. L. Welch, Efficient distributed recovery using message logging, *in* "Proceedings of the 8th ACM Symposium on Principles of Distributed Computing," pp. 223–238, 1989.
29. F. R. Soloway and P. A. Humblet, "On Distributed Network Protocols for Changing Topologies," Report LIDC-P-1564 MIT Laboratory for Information and Dec. Systems, May 1986.
30. R. E. Storm and S. A. Yemini, Optimistic recovery in distributed systems, *ACM Trans. Comput. Systems* **3** (1985), 204–226.
31. I. Yen and F. B. Bastani, A highly safe self-stabilizing mutual exclusion algorithm, *in* "Proceedings of the Second Workshop on Self-Stabilizing Systems," 1995.

YEHUDA AFEK, CTO and co-founder of WanWall, earned his Ph.D. in computer sciences from the University of California in 1985. He then joined the Distributed Systems Research Department of AT&T Bell Laboratories as a member of the technical staff. In 1988, Yehuda joined the Department of Computer Sciences at Tel Aviv University, where he now holds a tenured position as an associate professor.

SHLOMI DOLEV received his B.Sc. in engineering and B.A. in computer science in 1984 and 1985, and his M.Sc. and D.Sc. in computer science in 1990 and 1992 from the Technion Israel Institute of Technology. From 1992 to 1995 he was at Texas A&M University as a visiting research specialist. He visited Carleton University in 1994. In 1995 he joined the Department of Computer Science at Ben-Gurion University where he is now an associate professor. His current research interest include the theoretical aspects of distributed computing and communication networks; in particular the self-stabilization property of such systems.